# Predicción de volatilidad financiera

Apuntes teóricos y computacionales para la Semana 4

Limpieza, validación y documentación de datos financieros

**Objetivo de la semana.** Construir una versión limpia, ordenada y reproducible de la base histórica de `SPY` obtenida en la Semana 3. La limpieza incluirá la normalización del índice temporal, el tratamiento documentado de fechas duplicadas y valores faltantes, la validación de precios y volumen, la construcción de una bitácora de cambios y la exportación de una base procesada que pueda utilizarse en la Semana 5.

## Índice


**1. Contexto de la Semana 4** **2**

**2. Objetivos de aprendizaje** **2**

**3. Qué significa limpiar una base financiera** **3**

**4. Organización del notebook** **4**

**5. Lectura de la base original** **4**

**6. Creación de una copia de trabajo** **5**

**7. Bitácora de limpieza** **5**

**8. Normalización del índice temporal** **6**

**9. Ordenamiento cronológico** **7**

**10. Tratamiento de fechas duplicadas** **8**

10.1. Duplicados idénticos . . . . . 8

10.2. Fechas repetidas con valores distintos . . . . . 8

**11. Revisión de columnas esperadas** **9**

**12. Conversión de columnas numéricas** **10**

**13. Valores faltantes** **10**

13.1. Por qué no se imputan precios automáticamente . . . . . 11

**14. Días no bursátiles y fechas ausentes** **12**

**15. Validación de precios y volumen** **12**

15.1. Precios estrictamente positivos . . . . . 13

15.2. Relaciones OHLC . . . . . 13

15.3. Volumen no negativo . . . . . 14

15.4. Registro de la validación . . . . . 14

**16. Observaciones extremas** **14**

**17. Base de práctica con problemas artificiales** **15**

**18. Función de auditoría final** **16**

**19. Resumen de cambios** **17**

**20. Condiciones mínimas de aceptación** **18**

**21. Exportación de la base procesada** **18**

**22. Recuperación y validación del archivo limpio** **19**

**23. Pipeline completo sugerido** **20**

**24. Decisiones que deben documentarse** **21**

**25. Producto esperado al final de la semana** **21**

**26. Conclusión** **22**

# 1. Contexto de la Semana 4

En la Semana 3 se descargó la serie diaria de `SPY` desde Yahoo Finance para el periodo 2015–2025. La base original se guardó en

`data/raw/SPY_2015_2025_raw.csv`.

También se realizó una auditoría inicial de:

- columnas y tipos de datos;
- cobertura temporal;
- orden de las fechas;
- valores faltantes;
- fechas duplicadas;
- diferencias entre fechas consecutivas;
- consistencia elemental de precios y volumen.

La Semana 4 convierte ese diagnóstico en un procedimiento formal de limpieza. La secuencia general será

$$\text{base original} \longrightarrow \text{copia de trabajo} \longrightarrow \text{validación} \longrightarrow \text{base procesada}.$$

La base original no se modificará. Todas las transformaciones se aplicarán sobre una copia almacenada en memoria y el resultado se guardará en la carpeta `data/processed`.

**Delimitación de la semana.** En esta etapa todavía no se calcularán rendimientos logarítmicos, rendimientos absolutos, rendimientos al cuadrado ni volatilidades móviles. Estas transformaciones comienzan en las Semanas 5 y 6.

# 2. Objetivos de aprendizaje

Al finalizar la semana, la estudiante deberá ser capaz de:

1. leer de forma reproducible una base financiera guardada en CSV;
2. distinguir entre datos originales y datos procesados;
3. crear una copia de trabajo sin modificar la base original;
4. normalizar y ordenar un índice temporal;
5. identificar y clasificar fechas duplicadas;

6. distinguir entre duplicados idénticos y duplicados contradictorios;
7. cuantificar valores faltantes por fila y por columna;
8. justificar cuándo eliminar, recuperar o imputar una observación;
9. distinguir entre días no bursátiles y fechas faltantes;
10. validar relaciones básicas entre `Open`, `High`, `Low` y `Close`;
11. verificar positividad de precios y no negatividad del volumen;
12. construir una bitácora reproducible de limpieza;
13. comparar la base original con la base procesada;
14. exportar y volver a leer la base limpia;
15. comprobar mediante condiciones automáticas que el archivo final es apto para la siguiente etapa.

## 3. Qué significa limpiar una base financiera

La limpieza de datos no consiste simplemente en aplicar una instrucción automática para eliminar faltantes o duplicados. Una decisión de limpieza debe responder al menos cuatro preguntas:

1. ¿Qué problema se detectó?
2. ¿Cuál es su posible causa?
3. ¿Qué acción se aplicará?
4. ¿Cómo se documentará esa acción?

Sea

$$\mathcal{D}_{\text{raw}} = \{(t_i, X_i) : i = 1, \dots, n\}$$

la base original. El objetivo es construir una base procesada

$$\mathcal{D}_{\text{clean}} = \{(s_j, Y_j) : j = 1, \dots, m\}$$

que satisfaga condiciones mínimas de calidad:

- fechas válidas y ordenadas;
- una sola observación por fecha;
- ausencia de faltantes en las variables esenciales;
- precios estrictamente positivos;
- relaciones OHLC coherentes;
- volumen no negativo;

- transformaciones registradas.

No es obligatorio que $m = n$. Si se elimina una fila inválida, puede ocurrir que $m < n$. Sin embargo, cualquier diferencia debe quedar explicada en la bitácora.

> **Principio conservador.** En una serie financiera, una observación extrema puede representar un movimiento real del mercado. No debe eliminarse únicamente por ser grande o visualmente inusual.

## 4. Organización del notebook

El notebook de esta semana puede llamarse

`04_limpieza_validacion_datos.ipynb`.

Se utilizarán las bibliotecas siguientes:

```
import pandas as pd
import numpy as np
from pathlib import Path
```

Las rutas principales serán:

```
ruta_raw = Path("data/raw")
ruta_processed = Path("data/processed")

archivo_raw = ruta_raw / "SPY_2015_2025_raw.csv"
archivo_clean = ruta_processed / "SPY_2015_2025_clean.csv"
archivo_bitacora = ruta_processed / "bitacora_limpieza_SPY.csv"
```

La carpeta de salida se crea con:

```
ruta_processed.mkdir(parents=True, exist_ok=True)
```

## 5. Lectura de la base original

La base guardada en la Semana 3 se recupera mediante:

Antes de leer la base, conviene verificar que el archivo existe:

```
if not archivo_raw.exists():
    raise FileNotFoundError(
        f"No se encontro el archivo: {archivo_raw}"
    )
```

La base guardada en la Semana 3 se recupera mediante:

```
datos_raw = pd.read_csv(
    archivo_raw,
    index_col=0,
    parse_dates=True
)

if datos_raw.empty:
    raise ValueError("La base original esta vacia.")
```

Después se revisa su estructura:

```
print(datos_raw.shape)
print(datos_raw.columns)
print(type(datos_raw.index))
datos_raw.head()
```

La lectura del archivo no debe incluir transformaciones ocultas. En particular, no se deben eliminar filas durante la importación.

## 6. Creación de una copia de trabajo

La limpieza se realizará sobre una copia independiente:

```
datos = datos_raw.copy(deep=True)
```

La opción `deep=True` crea una copia de la estructura y de los datos contenidos en el `DataFrame`. De esta forma, las modificaciones aplicadas a `datos` no deben alterar `datos_raw`.

Puede verificarse con:

```
print("Mismo objeto:", datos is datos_raw)
print("Dimensiones raw:", datos_raw.shape)
print("Dimensiones copia:", datos.shape)
```

Se espera que `datos is datos_raw` sea `False`, pero que ambas tablas tengan inicialmente las mismas dimensiones.

> **Regla del proyecto.** El objeto `datos_raw` se utilizará únicamente como referencia. Todas las operaciones de limpieza se aplicarán al objeto `datos`.

## 7. Bitácora de limpieza

Cada modificación debe registrarse. Una estructura sencilla es una lista de diccionarios:

```
bitacora = []

```

```

def registrar_cambio(paso, problema, accion, filas_afectadas):
    bitacora.append({
        "Paso": paso,
        "Problema detectado": problema,
        "Accion aplicada": accion,
        "Filas afectadas": int(filas_afectadas)
    })
```

Por ejemplo:

```
registrar_cambio(
    paso="Lectura",
    problema="Ninguno",
    accion="Se creo una copia de trabajo",
    filas_afectadas=0
)
```

Al final del proceso, la lista se convertirá en una tabla:

```
bitacora_df = pd.DataFrame(bitacora)
bitacora_df
```

La bitácora debe indicar incluso cuándo una revisión no encontró problemas. Esto muestra que la validación fue realizada y no simplemente omitida.

## 8. Normalización del índice temporal

El índice debe representar fechas mediante un `DatetimeIndex`. Aunque `parse_dates=True` suele producir este tipo, se aplicará una normalización explícita:

```
datos.index = pd.to_datetime(
    datos.index,
    errors="coerce"
)
```

La opción `errors="coerce"` transforma una fecha no interpretable en `NaT`, que significa *Not a Time*.

El número de fechas inválidas se obtiene con:

```
fechas_invalidas = datos.index.isna().sum()
print("Fechas invalidas:", fechas_invalidas)
```

Si existen fechas inválidas, deben mostrarse antes de tomar una decisión. Como el índice no puede filtrarse mediante una columna ordinaria, puede utilizarse:

```
if fechas_invalidas > 0:
    print(datos.loc[datos.index.isna()])
```

En este proyecto, una fecha inválida impide interpretar la fila temporalmente. Por ello, después de documentarla, puede eliminarse:

```
if fechas_invalidas > 0:
    datos = datos.loc[~datos.index.isna()].copy()

registrar_cambio(
    paso="Indice temporal",
    problema="Fechas no interpretables",
    accion="Se eliminaron las filas con fecha invalida",
    filas_afectadas=fechas_invalidas
)
```

Si el índice tuviera información de zona horaria, puede retirarse para trabajar con fechas diarias simples:

```
if datos.index.tz is not None:
    datos.index = datos.index.tz_localize(None)
```

Finalmente, se asigna un nombre claro al índice:

```
datos.index.name = "Date"
```

## 9. Ordenamiento cronológico

Una serie temporal debe estar ordenada desde la fecha más antigua hasta la más reciente. Primero se registra el estado inicial:

```
estaba_ordenada = datos.index.is_monotonic_increasing
print("La base estaba ordenada:", estaba_ordenada)
```

Después se ordena:

```
datos = datos.sort_index()
```

La operación no elimina observaciones ni modifica sus valores; únicamente cambia el orden de las filas.

```
registrar_cambio(
    paso="Orden temporal",
    problema=(
        "Ninguno" if estaba_ordenada
        else "Las fechas no estaban ordenadas"
    ),
    accion="Se ordeno la base por fecha",
    filas_afectadas=(0 if estaba_ordenada else len(datos))
)
```

La verificación posterior es:

```
assert datos.index.is_monotonic_increasing
```

# 10. Tratamiento de fechas duplicadas

En una base diaria de un solo activo se espera una observación por fecha. Sin embargo, no todos los duplicados son equivalentes.

## 10.1. Duplicados idénticos

Dos filas son duplicados idénticos si tienen la misma fecha y los mismos valores en todas las columnas. Estas filas aportan información repetida y una de ellas puede eliminarse.

El número de filas completamente duplicadas se calcula con:

```
filas_duplicadas_exactas = datos.reset_index().duplicated().sum()
print(
    "Filas completamente duplicadas:",
    filas_duplicadas_exactas
)
```

La eliminación puede realizarse mediante:

```
if filas_duplicadas_exactas > 0:
    datos = (
        datos.reset_index()
        .drop_duplicates()
        .set_index("Date")
        .sort_index()
    )
```

## 10.2. Fechas repetidas con valores distintos

Después de retirar duplicados exactos, se revisa si todavía existen fechas repetidas:

```
mascara_fechas_duplicadas = datos.index.duplicated(
    keep=False
)

fechas_duplicadas = datos.loc[
    mascara_fechas_duplicadas
]

print(
    "Filas con fecha repetida:",
    len(fechas_duplicadas)
)
```

Si una misma fecha contiene valores distintos, no es adecuado conservar automáticamente la primera o la última observación. Tampoco se deben promediar los precios OHLC sin una justificación financiera.

La respuesta conservadora será detener el pipeline:

```
if not fechas_duplicadas.empty:
    display(fechas_duplicadas)
    raise ValueError(
        "Existen fechas duplicadas con valores distintos. "
        "Deben verificarse contra la fuente original."
    )
```

La bitácora se actualiza con:

```
registrar_cambio(
    paso="Duplicados",
    problema="Filas completamente duplicadas",
    accion="Se conservo una sola copia",
    filas_afectadas=filas_duplicadas_exactas
)
```

**Decisión metodológica.** En este proyecto solo se eliminarán automáticamente duplicados exactos. Las fechas repetidas con información contradictoria deberán revisarse en la fuente antes de continuar.

## 11. Revisión de columnas esperadas

La base debe contener las variables:

```
columnas_esperadas = [
    "Open",
    "High",
    "Low",
    "Close",
    "Adj Close",
    "Volume"
]
```

Se comparan con las columnas disponibles:

```
columnas_faltantes = [
    columna
    for columna in columnas_esperadas
    if columna not in datos.columns
]

columnas_extra = [
    columna
    for columna in datos.columns
    if columna not in columnas_esperadas
]

print("Columnas faltantes:", columnas_faltantes)
```

```
print("Columnas adicionales:", columnas_extra)
```

Si falta alguna variable esencial, el proceso debe detenerse:

```
if columnas_faltantes:
    raise ValueError(
        "Faltan columnas esenciales: "
        f"{columnas_faltantes}"
    )
```

Las columnas adicionales no se eliminan automáticamente. Primero debe identificarse su significado.

Para trabajar con un orden uniforme:

```
datos = datos[columnas_esperadas].copy()
```

## 12. Conversión de columnas numéricas

Las columnas financieras deben ser numéricas. Puede aplicarse:

```
for columna in columnas_esperadas:
    datos[columna] = pd.to_numeric(
        datos[columna],
        errors="coerce"
    )
```

Los valores que no puedan convertirse se transformarán en `NaN`. Por ello, esta conversión debe realizarse antes de la revisión definitiva de faltantes.

```
print(datos.dtypes)
```

Se espera que los precios tengan tipo de punto flotante y que el volumen sea numérico.

## 13. Valores faltantes

La tabla de faltantes se construye mediante:

```
reporte_faltantes = pd.DataFrame({
    "Valores faltantes": datos.isna().sum(),
    "Porcentaje": 100 * datos.isna().mean()
})

reporte_faltantes
```

También conviene identificar filas con al menos un faltante:

```
filas_con_faltantes = datos[datos.isna().any(axis=1)]
print("Filas con al menos un faltante:", len(filas_con_faltantes))
```

### 13.1. Por qué no se imputan precios automáticamente

Una regla como

```
datos = datos.ffill()
```

copiaría el precio anterior hacia la fecha faltante. Esto puede crear artificialmente un día sin cambio y modificar posteriormente los rendimientos y la volatilidad. De forma similar, una interpolación lineal introduciría un precio que no fue observado realmente.

Por ello, se seguirá el orden de decisión:

1. confirmar que el faltante se encuentra en una fecha de operación;
2. intentar recuperar el dato desde la fuente original;
3. si no puede recuperarse y la cantidad es pequeña, eliminar la fila completa;
4. documentar la acción;
5. evitar imputaciones automáticas en las columnas de precios.

Para la primera versión del proyecto, se considerarán esenciales todas las columnas OHLC, el cierre ajustado y el volumen:

```
columnas_esenciales = columnas_esperadas.copy()
numero_filas_faltantes = datos[
    columnas_esenciales
].isna().any(axis=1).sum()
```

Después de inspeccionar las filas afectadas, se eliminarán únicamente las filas que todavía contengan faltantes:

```
if numero_filas_faltantes > 0:
    display(
        datos[
            datos[columnas_esenciales]
            .isna()
            .any(axis=1)
        ]
    )

    datos = datos.dropna(
        subset=columnas_esenciales
    ).copy()
```

La decisión queda registrada:

```
registrar_cambio(
    paso="Valores faltantes",
    problema="Faltantes en columnas esenciales",
    accion=(
        "Se eliminaron filas despues de revisar "
```

```
        "que no fue posible recuperarlas"
    ),
    filas_afectadas=numero_filas_faltantes
)
```

Si la base real no contiene faltantes, el número de filas afectadas será cero.

## 14. Días no bursátiles y fechas ausentes

La ausencia de una fecha no significa necesariamente que falte un dato. `SPY` no opera regularmente durante fines de semana ni en ciertos días festivos.

Se revisan observaciones en sábado o domingo:

```
mascara_fin_semana = datos.index.dayofweek >= 5
numero_fin_semana = mascara_fin_semana.sum()

print(
    "Observaciones en fin de semana:",
    numero_fin_semana
)
```

Para `SPY`, una observación en fin de semana sería inusual. En lugar de eliminarla automáticamente, se mostrará y se detendrá la ejecución:

```
if numero_fin_semana > 0:
    display(datos.loc[mascara_fin_semana])
    raise ValueError(
        "Se encontraron observaciones en fin de semana. "
        "Verifique las fechas y la fuente."
    )
```

Las diferencias entre fechas consecutivas se obtienen con:

```
diferencias = datos.index.to_series().diff().dt.days
print(diferencias.value_counts().sort_index())
```

Una diferencia de tres días puede corresponder a viernes–lunes. Una diferencia mayor puede corresponder a un día festivo o a un cierre extraordinario. Para afirmar que falta una sesión de mercado se necesitaría comparar con un calendario bursátil, lo cual no forma parte de la limpieza básica de esta semana.

> **Regla importante.** No se crearán filas para sábados, domingos o festivos, ni se rellenarán precios para fechas en las que el mercado no operó.

## 15. Validación de precios y volumen

## 15.1. Precios estrictamente positivos

Los rendimientos logarítmicos de la Semana 5 requerirán precios positivos. Se revisa:

```
columnas_precios = [
    "Open",
    "High",
    "Low",
    "Close",
    "Adj Close"
]

mascara_precios_no_positivos = (
    datos[columnas_precios] <= 0
).any(axis=1)

numero_precios_no_positivos = (
    mascara_precios_no_positivos.sum()
)
```

Si aparecen casos:

```
if numero_precios_no_positivos > 0:
    display(datos.loc[mascara_precios_no_positivos])
    raise ValueError(
        "Existen precios no positivos. "
        "No se pueden calcular logaritmos."
    )
```

## 15.2. Relaciones OHLC

Para cada fecha $t$ se espera:

$$L_t \leq H_t,$$
$$L_t \leq O_t \leq H_t,$$
$$L_t \leq C_t \leq H_t.$$

La máscara de inconsistencias es:

```
mascara_ohlc = (
    (datos["High"] < datos["Low"])
    | (datos["Open"] < datos["Low"])
    | (datos["Open"] > datos["High"])
    | (datos["Close"] < datos["Low"])
    | (datos["Close"] > datos["High"])
)

numero_inconsistencias_ohlc = mascara_ohlc.sum()
```

En caso de encontrar inconsistencias:

```
if numero_inconsistencias_ohlc > 0:
    display(datos.loc[mascara_ohlc])
    raise ValueError(
        "Existen inconsistencias OHLC. "
        "Deben verificarse contra la fuente."
    )
```

### 15.3. Volumen no negativo

```
mascara_volumen_negativo = datos["Volume"] < 0
numero_volumen_negativo = mascara_volumen_negativo.sum()

if numero_volumen_negativo > 0:
    display(datos.loc[mascara_volumen_negativo])
    raise ValueError(
        "Existen observaciones con volumen negativo."
    )
```

Un volumen igual a cero no se eliminará automáticamente. Primero deberá verificarse si corresponde a la fuente o a una situación particular del mercado.

### 15.4. Registro de la validación

```
registrar_cambio(
    paso="Validacion financiera",
    problema="Precios, relaciones OHLC y volumen",
    accion="Se verificaron las condiciones basicas",
    filas_afectadas=(
        numero_precios_no_positivos
        + numero_inconsistencias_ohlc
        + numero_volumen_negativo
    )
)
```

## 16. Observaciones extremas

Una base puede contener precios o volúmenes muy alejados de sus valores habituales. Estas observaciones pueden deberse a:

- movimientos reales del mercado;
- eventos corporativos;
- errores de descarga;
- problemas en la fuente;

- unidades o formatos incorrectos.

En esta semana no se aplicará una regla automática para eliminar extremos. Solo se identificarán valores potencialmente inusuales mediante estadísticas descriptivas:

```
datos.describe().T
```

También puede revisarse el volumen mediante cuantiles:

```
datos["Volume"].quantile(
    [0.50, 0.75, 0.90, 0.95, 0.99, 1.00]
)
```

La presencia de una cola larga en el volumen no implica que la base sea incorrecta.

## 17. Base de práctica con problemas artificiales

Es posible que la base real de `SPY` no contenga faltantes, duplicados o desorden. Para practicar las operaciones sin alterar el archivo original, se construirá una copia artificial.

```
practica = datos_raw.head(12).copy()

# 1. Desordenar las fechas
practica = practica.sample(
    frac=1,
    random_state=123
)

# 2. Introducir dos valores faltantes
practica.iloc[2, practica.columns.get_loc("Close")] = np.nan
practica.iloc[5, practica.columns.get_loc("Volume")] = np.nan

# 3. Agregar una fila duplicada exacta
fila_repetida = practica.iloc[[3]].copy()
practica = pd.concat([practica, fila_repetida])
```

Esta base debe utilizarse únicamente para responder:

1. ¿cómo se detecta el desorden temporal?;
2. ¿cómo se localizan los faltantes?;
3. ¿cómo se elimina un duplicado exacto?;
4. ¿cómo cambia el número de filas después de la limpieza?;
5. ¿cómo se documentan las decisiones?

Después de practicar, el pipeline definitivo se ejecutará nuevamente desde `datos_raw`, no desde `practica`.

**Precaución.** La base artificial no debe guardarse con el nombre del archivo limpio. Su objetivo es exclusivamente pedagógico.

## 18. Función de auditoría final

Las verificaciones principales pueden reunirse en una función:

```python
def auditar_base_financiera(df):
    columnas = [
        "Open",
        "High",
        "Low",
        "Close",
        "Adj Close",
        "Volume"
    ]

    reporte = {
        "Numero de filas": len(df),
        "Numero de columnas": df.shape[1],
        "Indice temporal": isinstance(
            df.index,
            pd.DatetimeIndex
        ),
        "Fechas ordenadas": (
            df.index.is_monotonic_increasing
        ),
        "Fechas unicas": df.index.is_unique,
        "Total de faltantes": int(
            df[columnas].isna().sum().sum()
        ),
        "Precios no positivos": int(
            (df[[
                "Open", "High", "Low",
                "Close", "Adj Close"
            ]] <= 0).sum().sum()
        ),
        "Volumen negativo": int(
            (df["Volume"] < 0).sum()
        ),
        "Inconsistencias OHLC": int((
            (df["High"] < df["Low"])
            | (df["Open"] < df["Low"])
            | (df["Open"] > df["High"])
            | (df["Close"] < df["Low"])
            | (df["Close"] > df["High"])
        ).sum()),
        "Observaciones en fin de semana": int(
            (df.index.dayofweek >= 5).sum()
        )
```

```
    }

    return pd.Series(
        reporte,
        name="Resultado"
    ).to_frame()
```

La función se aplica antes y después de la limpieza:

```
auditoria_raw = auditar_base_financiera(datos_raw)
auditoria_clean = auditar_base_financiera(datos)
```

Para comparar ambos resultados:

```
comparacion_auditoria = pd.concat(
    [auditoria_raw, auditoria_clean],
    axis=1
)

comparacion_auditoria.columns = [
    "Base original",
    "Base limpia"
]

comparacion_auditoria
```

## 19. Resumen de cambios

Se registran las dimensiones y fechas antes y después del proceso:

```
resumen_limpieza = pd.DataFrame({
    "Elemento": [
        "Filas originales",
        "Filas procesadas",
        "Filas eliminadas",
        "Columnas originales",
        "Columnas procesadas",
        "Primera fecha original",
        "Ultima fecha original",
        "Primera fecha procesada",
        "Ultima fecha procesada"
    ],
    "Resultado": [
        len(datos_raw),
        len(datos),
        len(datos_raw) - len(datos),
        datos_raw.shape[1],
        datos.shape[1],
        datos_raw.index.min(),
        datos_raw.index.max(),
```

```
        datos.index.min(),
        datos.index.max()
    ]
})

resumen_limpieza
```

La tabla permite responder de forma transparente si el proceso cambió el tamaño o la cobertura de la base.

## 20. Condiciones mínimas de aceptación

Antes de guardar la base limpia se verificarán las siguientes condiciones:

```
assert isinstance(
    datos.index,
    pd.DatetimeIndex
)
assert datos.index.is_monotonic_increasing
assert datos.index.is_unique
assert datos[columnas_esenciales].isna().sum().sum() == 0
assert (datos[columnas_precios] > 0).all().all()
assert (datos["Volume"] >= 0).all()
assert not mascara_ohlc.any()
assert (datos.index.dayofweek < 5).all()
```

Si todas las instrucciones se ejecutan sin error, la base cumple los criterios básicos definidos para el proyecto.

Estas condiciones no demuestran que cada observación sea correcta, pero evitan continuar con errores estructurales evidentes.

## 21. Exportación de la base procesada

La base limpia se guardará en:

`data/processed/SPY_2015_2025_clean.csv`.

El código es:

```
datos.to_csv(
    archivo_clean,
    index=True,
    date_format="%Y-%m-%d"
)

print("Base limpia guardada en:", archivo_clean)
```

La bitácora se guarda por separado:

```
bitacora_df = pd.DataFrame(bitacora)
bitacora_df.to_csv(
    archivo_bitacora,
    index=False
)
```

También puede guardarse el resumen:

```
archivo_resumen = (
    ruta_processed
    / "resumen_limpieza_SPY.csv"
)

resumen_limpieza.to_csv(
    archivo_resumen,
    index=False
)
```

**Nombres distintos.** El archivo `raw` y el archivo `clean` deben conservar nombres diferentes. El segundo nunca debe sobrescribir al primero.

## 22. Recuperación y validación del archivo limpio

Después de guardar, se vuelve a leer el archivo:

```
datos_verificados = pd.read_csv(
    archivo_clean,
    index_col="Date",
    parse_dates=True
)
```

Se comparan dimensiones, columnas e índice:

```
assert datos_verificados.shape == datos.shape
assert datos_verificados.columns.tolist() == datos.columns.tolist()
assert datos_verificados.index.equals(datos.index)
```

Para una comparación numérica robusta puede utilizarse:

```
pd.testing.assert_frame_equal(
    datos_verificados,
    datos,
    check_freq=False,
    check_dtype=False,
    rtol=1e-10,
    atol=1e-12
)
```

La tolerancia considera que el formato CSV puede producir diferencias numéricas muy pequeñas al escribir y recuperar números de punto flotante.

Finalmente:

```
print(
    "El archivo limpio se guardo y recupero "
    "correctamente."
)
```

## 23. Pipeline completo sugerido

El procedimiento de la semana puede resumirse como:

1. verificar la existencia del archivo original;
2. leer `SPY_2015_2025_raw.csv`;
3. crear una copia profunda;
4. iniciar la bitácora;
5. convertir el índice a fechas;
6. retirar fechas inválidas, si existieran;
7. ordenar cronológicamente;
8. eliminar duplicados exactos;
9. detenerse ante duplicados contradictorios;
10. verificar las columnas esperadas;
11. convertir columnas a tipo numérico;
12. revisar valores faltantes;
13. eliminar únicamente filas esenciales no recuperables;
14. revisar observaciones en fin de semana;
15. validar precios, OHLC y volumen;
16. construir la auditoría final;
17. comparar base original y base limpia;
18. ejecutar las condiciones de aceptación;
19. guardar base, bitácora y resumen;
20. volver a leer el archivo y verificarlo.

## 24. Decisiones que deben documentarse

La conclusión de la semana debe incluir una tabla como la siguiente:

| **Problema** | **Resultado** | **Decisión** |
|---|---|---|
| Fechas inválidas | Completar con Python | Eliminar o revisar. |
| Orden temporal | Completar con Python | Ordenar de forma ascendente. |
| Duplicados exactos | Completar con Python | Conservar una sola copia. |
| Duplicados contradictorios | Completar con Python | Verificar en la fuente. |
| Valores faltantes | Completar con Python | Recuperar o eliminar la fila. |
| Fines de semana | Completar con Python | Verificar, no eliminar automáticamente. |
| Precios no positivos | Completar con Python | Detener y revisar. |
| Inconsistencias OHLC | Completar con Python | Detener y revisar. |
| Volumen negativo | Completar con Python | Detener y revisar. |

La expresión "no se encontraron problemas" es una conclusión válida cuando está respaldada por las verificaciones correspondientes.

## 25. Producto esperado al final de la semana

La estudiante deberá entregar:

1. el notebook `04_limpieza_validacion_datos.ipynb`;
2. la base original sin modificaciones;
3. el archivo `data/processed/SPY_2015_2025_clean.csv`;
4. el archivo `data/processed/bitacora_limpieza_SPY.csv`;
5. el archivo `data/processed/resumen_limpieza_SPY.csv`;
6. la tabla de auditoría antes y después de la limpieza;
7. la tabla con las decisiones adoptadas;
8. una conclusión breve sobre la calidad final de la base;
9. la solución de la práctica con datos artificialmente alterados.

La conclusión deberá indicar:

- cuántas filas tenía la base original;
- cuántas filas tiene la base limpia;
- si se encontraron faltantes o duplicados;

- si se modificó la cobertura temporal;
- si las condiciones OHLC fueron satisfechas;
- si la base puede utilizarse para calcular rendimientos.

## 26. Conclusión

En esta semana se construyó un procedimiento reproducible para transformar la base original de `SPY` en una base procesada. La limpieza no se planteó como una colección de instrucciones automáticas, sino como una secuencia de diagnóstico, decisión, validación y documentación.

El índice temporal fue normalizado y ordenado. Los duplicados exactos pueden eliminarse conservando una sola copia, mientras que las fechas repetidas con valores contradictorios deben revisarse contra la fuente. Los valores faltantes en variables esenciales no se imputan automáticamente, porque una imputación puede alterar los rendimientos y la volatilidad calculados posteriormente.

También se verificaron condiciones básicas de calidad financiera: precios positivos, coherencia entre apertura, máximo, mínimo y cierre, volumen no negativo y ausencia de observaciones bursátiles ordinarias en fin de semana. Las decisiones quedaron registradas en una bitácora y el archivo procesado se guardó con un nombre distinto al archivo original.

El producto principal de la semana es

`data/processed/SPY_2015_2025_clean.csv`,

que servirá como entrada para la Semana 5. En esa etapa se calcularán los rendimientos logarítmicos, los rendimientos absolutos y los rendimientos al cuadrado, sin volver a modificar la base original.

### Bibliografía mínima sugerida


- McKinney, W. *Python for Data Analysis*. O'Reilly Media.
- Tsay, R. S. *Analysis of Financial Time Series*. Wiley.
- Brooks, C. *Introductory Econometrics for Finance*. Cambridge University Press.
- `pandas`. Documentación oficial: https://pandas.pydata.org/docs/.